

AI-Assisted Summary Generation Methods

This section describes the automated pipeline developed to generate structured best-practice guidance summaries from clinical trial documents using a large language model (LLM). Figure 1 provides an end-to-end overview of the pipeline architecture.

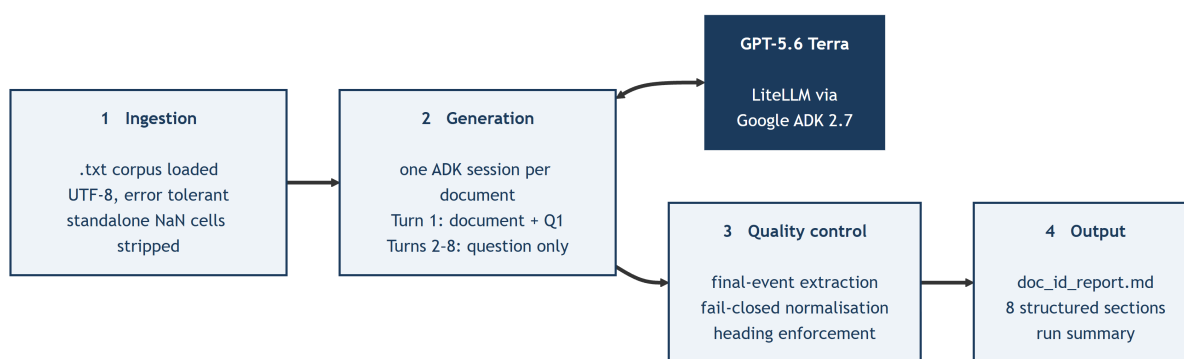


Figure 1: Overview of the AI-assisted summary generation pipeline. Clinical trial guidance documents are loaded and normalized (left), processed through GPT-5.6 Terra via a two-tier prompting architecture in which the source text is supplied once per document session (center), passed through three automated quality-control steps (right), and saved as structured Markdown reports (far right). Documents are processed concurrently; a transient provider failure retries the affected document against a fresh session, and a document that still fails is recorded as such without interrupting the remainder of the batch.

Dataset Preparation

The input corpus consisted of clinical trial guidance documents in plain-text format. Each document addressed a distinct clinical trial practice, encompassing its definition, rationale, implementation protocols, evaluation criteria, and alternative approaches. Documents were loaded programmatically from a designated directory and indexed by filename identifier, enabling systematic tracking throughout the pipeline. Text encoding was normalized to UTF-8 with error-tolerant character handling to accommodate potential encoding artifacts. The source documents were sufficiently well-formed for direct model ingestion.

Model Selection and Configuration

Summary generation was performed using OpenAI’s GPT-5.6 Terra, a large language model with a 1.05-million-token context window and strong long-context comprehension and instruction-

following capabilities. The context window comfortably accommodated the largest document in the corpus (approximately 25,000 tokens), so no document chunking was required. The model was accessed through the LiteLLM abstraction layer integrated within the Google Agent Development Kit (ADK, version $\geq 2.7.1$), which provides a runner architecture with an in-memory session service for managing session state, message construction, and streamed response handling. Default inference parameters were used throughout, as output quality and structural consistency were governed by the prompting strategy detailed below. The LiteLLM integration additionally permits execution with alternative model providers, facilitating cross-model reproducibility assessment.

Prompting Strategy

A two-tier prompting architecture was designed to enforce consistent, document-grounded output across all processed documents.

The *system-level* prompt established persistent behavioral constraints, specifying the model’s role as a best-practice guidance report writer operating under four non-negotiable rules:

- (1) exclusive reliance on the provided document content, with no recourse to external knowledge, common-sense inference, or assumption;
- (2) a fail-closed policy mandating the verbatim output “Not found in document” whenever the source material did not explicitly address a query or subsection;
- (3) adoption of a professional, hierarchical guidance-report tone; and
- (4) use of a standardized set of subsection labels—including “Key messages,” “Definition and purpose,” “How to implement,” and “How to evaluate”—to ensure structural uniformity.

At the *user-message* level, a parameterized template was instantiated for each query. The complete document text was supplied once, in the opening turn of each document’s session, together with the first question; the seven subsequent turns carried only the question and a restatement of the document-only constraint, since the session already retained the source text. This design avoids re-transmitting the document with every question, which would otherwise multiply input-token consumption by the number of questions posed. Eight standardized

questions were posed for every document:

1. What is the definition of this practice?
2. Why is this practice beneficial or important?
3. When should this practice be used?
4. When is it justified to not use this practice?
5. How should the correct methods be selected?
6. How should the correct methods be implemented?
7. How should the correct methods be evaluated?
8. If it is justified not to use this practice, what should be done as an alternative?

Each prompt directed the model to open its response with a standardized Markdown heading (e.g., ## Q1: What is the definition of this practice?) and to populate only those subsection labels applicable to the given question. This dual-layer design—persistent system constraints paired with per-question structural directives—constituted the primary mechanism for cross-document consistency.

Generation Procedure and Quality Controls

The processing pipeline was implemented as a fully asynchronous program. Documents were processed concurrently up to a configurable limit (default: three) regulated by an asynchronous semaphore, while the eight questions within each document were posed sequentially. Sequential questioning preserved conversational session context within the ADK framework, enabling the model to accumulate topical awareness across questions for the same document. Each document was assigned an isolated in-memory session to prevent cross-document information leakage.

Three automated post-processing steps were applied to every model response:

1. *Event extraction.* The ADK runner’s streamed event objects were filtered to the agent’s final response, excluding partial and intermediate events, and the retained text segments were concatenated into a single coherent response.
2. *Fail-closed normalization.* Empty or null outputs were replaced with the canonical fallback string “Not found in document,” ensuring completeness.

3. *Heading enforcement.* The expected Markdown heading was verified and, if absent, prepended automatically to guarantee uniform report structure.

Two mechanisms governed provider failures. First, transient errors—rate limiting and server-side faults—triggered a bounded retry with exponential backoff and jitter, applied to the document as a whole rather than to the individual question. The ADK runner commits a question to the session before the model responds, so retrying a single question would enter it into the conversation twice; each retry therefore rebuilt the runner and session and re-posed all eight questions. Second, a document that exhausted its retries was recorded as failed and excluded from the output set while the remaining documents continued to completion, with the run terminating in a non-zero status and an enumeration of the affected documents.

Final reports were saved as individual Markdown files following the naming convention `{document_identifier}_report.md`. An overwrite-control flag permitted the pipeline to bypass previously completed documents on subsequent runs, eliminating redundant model invocations.

Reproducibility Considerations

The full pipeline—including all prompt texts, system instructions, and post-processing logic—was maintained under Git version control. Runtime configuration parameters (model identifier, directory paths, concurrency limits, and overwrite behavior) were externalized to an environment variable file, decoupling configuration from source code. Runtime parameters were additionally exposed as command-line options that override the environment, so a reported run is reproducible from its invocation alone, and exact dependency versions were pinned in a lockfile. Code quality was enforced through automated linting and formatting with Ruff, static type checking with mypy in strict mode, and an automated test suite executed on every change under continuous integration; the suite substitutes a stub runner for the model and therefore issues no network requests. While the deterministic components of the pipeline (prompt construction, post-processing, file output) are fully reproducible, large language model inference is inherently stochastic. The standardized prompting framework and structural post-processing serve to bound the practical impact of this variability on report structure and content coverage.